

Chapitre 3: Gestion de code source avec Git/GitHub



Plan

- I. Introduction
- II. La gestion de version
- III. Installation de Git et manipulation des commandes de base
- IV. Méthode de travail en équipe avec Git/GitHub

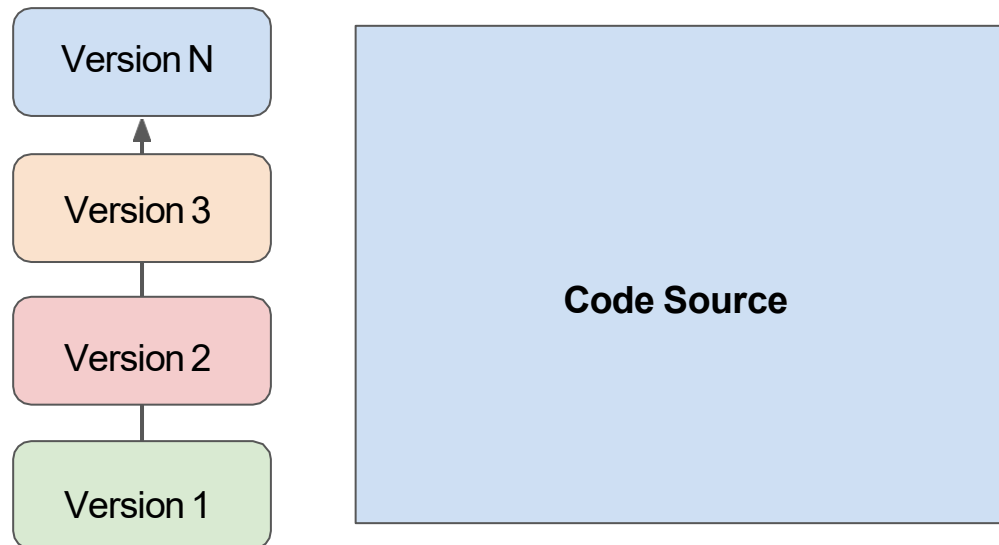
I. Introduction

Git et **GitHub** sont devenus des outils incontournables dans le développement logiciel moderne. Ils permettent de gérer efficacement le code source, de suivre les modifications, et de faciliter le travail collaboratif au sein des équipes.

Git : est un système de contrôle de version distribué qui permet à chaque développeur de disposer d'une copie complète du projet. Il sert à enregistrer, suivre et organiser les changements du code au fil du temps.

GitHub : est une plateforme en ligne qui s'appuie sur Git. Elle permet aux développeurs de travailler ensemble plus facilement, en partageant leurs projets, en proposant des améliorations, et en suivant les tâches à réaliser ou les problèmes à corriger.

II. La gestion de version c'est quoi?



- Sauvegarder l'évolution du code
- Pouvoir revenir en arrière et comparer des versions
- Voir et comprendre l'évolution du code
- Travailler à plusieurs sur un projet
- Pouvoir se tromper sans impacter les autres

III. Installation et Manipulation des commandes de bases de GIT

1. INSTALLATION DE GIT

L'installation de l'outil Git peut se faire via un terminal ou avec une interface graphique de base selon l'OS de votre ordinateur.

Linux: `sudo apt install git-all`

Mac: <https://git-scm.com/download/mac>

Windows: <https://git-scm.com/download/win>

2. CREATION DE COMPTE SUR GITHUB

Pour créer un compte sur GitHub, il faut suivre le lien suivant: <https://github.com/>

3. CONFIGURATION

Nous allons donc commencer à ajouter notre **username** ainsi que notre **adresse email** ce qui permettra d'identifier l'auteur des commits (les modifications apportées sur le projet). Pour cela, nous allons procéder comme suit:

- a) Définir le username de l'utilisateur :
git config --global user.name "username"
- b) Définir l'email de l'utilisateur :
git config --global user.email email

4. GESTION DES DÉPÔTS

Un dépôt (référentiel ou repository), est une collection de fichiers et de leur historique de mise à jour. C'est là que vit votre projet et où Git stocke l'historique des modifications.

a) Pour que notre projet local soit un dépôt git , on doit donc l'initialiser en tapant la commande : **git init**

git init est donc la première commande qu'on doit taper au niveau de chaque projet si on souhaite initialiser notre projet en tant que git. Cette commande crée un dossier caché nommé **.git** , c'est dans ce dossier caché que git va faire toutes ses configurations.

b) Après avoir créer le dépôt local , il faut aller sur GitHub, créer le dépôt distant (repository) ayant même nom que le dépôt local .

5. GÉRER LES MODIFICATIONS

Pour voir les changements de notre projet local, les sélectionner et les enregistrer, on effectue les étapes suivantes:

a) Lister tous changements apportés à l'espace de travail(connaitre le statut actuel de votre projet) :

git status

b) Ajouter les modifications de fichiers spécifiques dans votre projet local(dépôt git local) :

git add nom_du_fichier ou
git add --all

c) Enregistrer les modifications présentes dans votre projet local :

git commit -m "message"

d) Pour interconnecter le dépôt local avec le dépôt distant(ajouter un dépôt distant à un dépôt Git local), utilisez la commande suivante:

git remote add origin url dépôt_distant

La commande **git remote add origin url dépôt_distant** permet d'établir un lien entre notre dépôt local et notre dépôt distant, ce qui nous permet de pousser(push) et de tirer(pull) des modifications entre les deux.

origin : est le nom par défaut utilisé pour désigner le dépôt distant principal d'un projet git.

7. SYNCHRONISER LES CHANGEMENTS

Pour synchroniser le dépôt local et distant., on précède comme suit:

- a) Mettre à jour son dépôt local avec les commits présents sur le dépôt distant:

git pull origin nom_branche

- b) Envoyer les commits en local sur le dépôt distant:

git push origin nom_branche

6. TRAVAILLER EN BRANCHE

Nous allons voir comment manipuler une branche de travail.

Les branches nous permettent de travailler sur différentes fonctionnalités ou corrections de notre projet de manière indépendante. Il existe une branche par défaut appelée branche **master**. Elle est souvent appelée aussi branche **main**.

a) Lister toutes les branches :

git branch -a

b) Créer une nouvelle branche :

git branch nom_branche

c) Supprimer une branche :

git branch -d nom_branche

d) Créer une branche et s'y déplacer en même temps :

git checkout -b nom_branche

e) Pour fusionner une branche de travail (comme **test**) qui est en avance sur la branche principale (**master**), vous pouvez suivre les étapes ci-dessous. Cela suppose que vous avez déjà vérifié et testé vos modifications sur la branche **test** :

- Basculer(se déplacer) sur la branche **master** :

git checkout master

- Fusionner la branche **test** dans **master** :

git merge test

- Envoyer la fusion (le merge) sur la branche **master**:

git push origin master

III. Méthode de travail en équipe avec Git/GitHub

Travailler en équipe avec Git et GitHub implique plusieurs pratiques et méthodes pour assurer une collaboration efficace et éviter les conflits. Voici une méthode de travail en équipe avec Git/GitHub qui est couramment utilisée :

- 1. Création du dépôt GitHub** : Le projet commence généralement par la création d'un dépôt sur GitHub. Assurez-vous de choisir un nom significatif et une description claire pour le dépôt.
- 2. Clonage du dépôt** : Chaque membre de l'équipe doit cloner le dépôt sur sa machine locale en utilisant la commande: **git clone url_dépot_distant**

Cela crée une copie locale du dépôt distant sur l'ordinateur de chaque membre.

- 3. Branches** : Au lieu de travailler directement sur la branche principale (habituellement master ou main), chaque développeur doit créer une branche dédiée pour travailler sur une fonctionnalité ou une tâche spécifique. Cela se fait avec la commande:
- git checkout -b nom_branche**

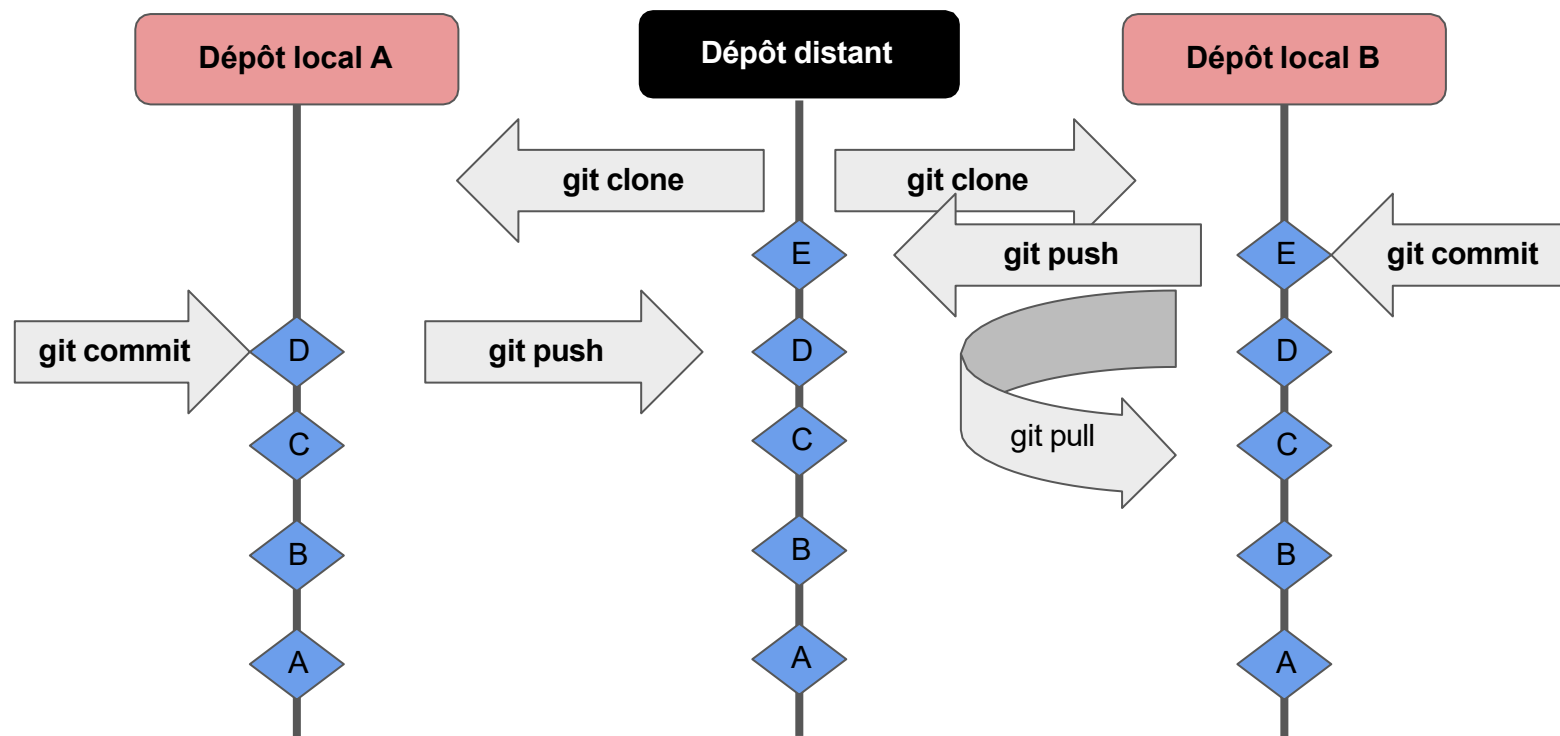
L'utilisation de branches permet d'isoler les modifications en cours de développement et de les fusionner plus tard de manière sécurisée.

- 3. Développement** : Chaque développeur travaille sur sa propre branche, en effectuant des commits réguliers pour enregistrer ses modifications. Il est recommandé de faire des commits atomiques, c'est-à-dire des commits qui ne concernent qu'une seule modification logique.

5. Résolution des conflits : Parfois, lorsque plusieurs développeurs modifient le même fichier, des conflits peuvent survenir lors de la fusion des branches. Il est important de les résoudre correctement en examinant les différences et en prenant des décisions appropriées pour intégrer les modifications.

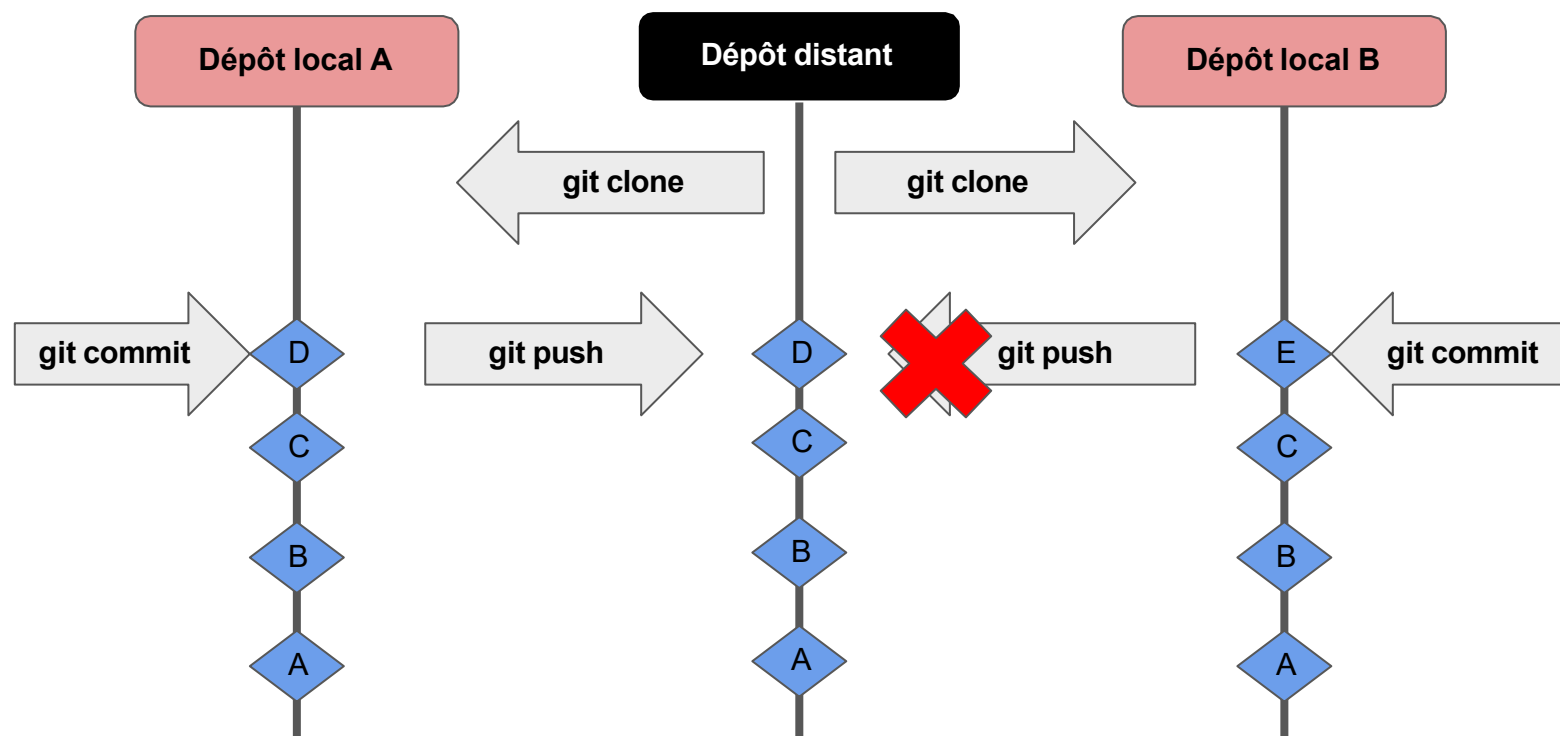
Pour éviter les conflits , voici les meilleures pratiques à adopter:

Sans conflits



voici les mauvaises pratiques qui peuvent amener des conflits :

Avec conflits



6. Code Review : Avant de fusionner une branche dans la branche principale, il est recommandé d'effectuer une revue de code. Les membres de l'équipe passent en revue le code pour repérer les erreurs, s'assurer du respect des normes de codage et partager leurs commentaires. Cela peut être fait à l'aide de fonctionnalités intégrées de GitHub telles que les **pull requests** .

7. Fusion (merge) : Une fois que la revue de code est terminée et que toutes les modifications ont été validées, la branche de développement peut être fusionnée dans la branche principale à l'aide de la commande: **git merge nom_branche**

ou bien d'une **pull request sur GitHub**.

Une **pull request sur GitHub** est un processus couramment utilisé pour fusionner(merger) des modifications d'une autre branche dans la branche principale d'un projet.

8. Les étapes pour créer une Pull Request :

- Allez sur la page GitHub du dépôt original(dépôt avec la branche master).
- Vous verrez un bouton "**Compare & pull request**" après avoir poussé vos modifications. Cliquez dessus.
- Remplissez le formulaire de pull request avec une description détaillée de vos modifications et cliquez sur "**Create pull request**".